octava

User manual

Table of contents

Introduction 3
Installation 4
    Docker installation 4
    Conda installation 5
Resource requirements 6
Setting up a dataset 7
    Input files and options 7
Edit dataset 11
Phenotype/sample view 12
Association analysis 13
    Genotype data preparation 13
    Association analysis options 14
    Models 14
    Association analysis results 15
Gene / variant view 17
    Input options 17
    Results 18

Introduction

Octava is an application for the analysis of genetic variation and functional annotation. It is composed and designed as a pipeline:

1. The starting point are raw genotype and phenotype data;
2. The input can be visualized and processed by `filtering or transforming phenotype data` and by `filtering, imputing, phasing genotype data` , as well as constructing haplotype blocks;
3. Processed genotype and phenotype data can be used for `association analysis (GWAS)` using various models, with support for relatedness correction;
4. GWAS results in the form of `Manhattan plots` can be visualized and compared;
5. Loci or markers of interest can be studied on the functional side, analyzing and visualizing the `variant and gene annotation, variant impact on proteins, gene expression` or gene localization and features.

However, **all (or most) parts of the application are independent and can be used separately**, as well as when some input files are not provided.

Octava runs as a local instance on a computer, with an **interface accessible through a web browser**. It is an **assembly of existing tools** and applications widely used in bioinformatics and genetic analysis, implemented in a single toolbox with an easy-to-use interface to the user. The main tools implemented are:

- `PLINK2` for genotype data processing and filtering;
- `PLINK1` for haplotype block construction;
- `BEAGLE` for genotype data imputation and phasing;
- `GEMMA`, `BLINK`, `GCTB`, `Regenie` and `Random forest` for association analysis;
- `SnpEff` for variant annotation;
- `PROVEAN` for variant severity prediction;
- `IGV.js` for genome visualization;
- `Plotly.js` for plotting and visualization.

As input, it takes several files in formats standardly used in bioinformatics analysis, and supporting files in text format (see below). It supports the analysis of an arbitrary **diploid** genome, and can accommodate an arbitrary number of genomes at once.

Octava is released under the GNU General Public License v3.0.

Installation

Octava is provided as both a **Docker image** and a **Conda environment**, which create an isolated environment that contains all the requirements for running the app.

Docker installation

1. Create a directory on your system (later referred to as `data` directory). This will contain all your input files (each genome dataset in a separate folder).
2. Install Docker desktop on your computer: follow the instructions for Windows, MacOS, Linux.
   - Requirements: Windows 10 or higher with WSL2 enabled; MacOS 11 (Big Sur) or newer; Linux 64-bit distribution with KVM virtualization
   - On Windows: to install WSL, open PowerShell and run `wsl --install` (instructions)
3. Navigate to the folder containing the Octava Docker image (`octava.tar.gz` or a similar name). No need to open or unzip the file.
4. Open command prompt in this folder:
   - On Windows, click the address bar (top of the file explorer), write `cmd` and press `↵Enter` .
   - On MacOS, double-click the folder and select `New Terminal at Folder` .
   - Or use the `cd` command in the command prompt to change directory.
5. Within the command prompt, write `docker image load -i octava.tar.gz` , press `↵Enter` to run (or replace `octava.tar.gz` with the name of the source file).
6. Installation will take several minutes and require up to 10GB of free storage.
7. After installation, open the Docker desktop application.
8. Go to the `Images` tab, select the `octava` image and click `▶(Run)` .
9. In the dialog window, expand `Optional settings` , enter any name. Under `Ports` , enter 5101 (unless you have a reason to use a different port).
10. Under `Volumes` , click `…` and navigate to the `data` directory from step 1. Open the folder and click `Select folder` . Under `Container path` , enter `/octava/data/` .
11. Add a line under `Volumes` by clicking `+` , click `…` and navigate to the folder that will contain your protein BLAST database (see below under **Setting up a dataset**). `Select folder` and under `Container path` , enter `/octava/prot_db/` .
12. Click `▶(Run)` .
13. You will access your application in your web browser, at `localhost:5101` (or your custom port).
14. You can stop the application under `Containers` tab on the right, by clicking `■(Stop)` .
15. To start Octava again, go to `Containers` and start it using `▶(Run)` , without repeating any of the steps above.

The application will **update to the latest version** every time it is restarted. To prevent automatic updates, create a file called `.noupdate` in the `data` directory.

Conda installation

1. **If you are using Windows and you do not have Ubuntu terminal installed:**
   - Open PowerShell and run `wsl --install Ubuntu` .
   - Open Ubuntu from the start menu. You will be asked to choose a username and a password.
   - Back in PowerShell, set the default user: `wsl --manage Ubuntu --set-default-user <user>` (replace `<user>` with your chosen username from the step above).
   - Restart your Ubuntu terminal.
   - Run `sudo apt update && sudo apt install -y build-essential unzip` to install required packages. You will be asked to provide the password from the step above.
2. **If you do not have conda installed:**
   - Download Conda as your desired distribution, such as MiniForge (download links).
   - In Ubuntu (Windows) or Terminal (Linux/Mac OS), navigate to the directory containing your Conda file (e.g. Miniforge3-Linux-x86_64.sh) and run `bash Miniforge3-Linux-x86_64.sh` .
   - You will be asked to provide a directory to install conda into. You can keep it as the suggested path (recommended), or change it. Make note of this path. Finish conda setup by running `echo 'export PATH="<conda_path>/bin:$PATH"' >> ~/.bashrc` , replace `<conda_path>` with your path. Restart Ubuntu.
   - Run `conda init` to initialize the conda installation.
3. Create a folder that will contain your application and all input files.
4. Place `octava_conda.tar.gz` within, navigate to the folder within Ubuntu/Terminal.
5. Run `tar -xzf octava_conda.tar.gz` to extract the contents.
6. Run `./conda_setup.sh` to install the app.
7. Place all your genome input files into `octava/data` (each genome dataset in separate folder).
8. Optional: place your BLAST database (see **Setting up a dataset**) into `octava/prot_db` .
9. Change working directory to `octava` , activate the environment using `conda activate octava` .
10. Start the app by running `./start.sh` .
11. To stop the application, press `CTRL+C` and run `conda deactivate` .
12. To start Octava again, repeat steps `9` and `10` .

The application will **update to the latest version** every time it is restarted. To prevent automatic updates, create a file called `.noupdate` in the `data` directory.

Resource requirements

Storage

Octava supports an arbitrary number of datasets concurrently set up for analysis. Apart from the input files provided by the user, additional files will be created. Make sure you have sufficient storage for each dataset added. A rule of thumb is to allocate storage two to three times as much as your input VCF file.

Processing power

Processing power should not be a limiting factor for analysis. With less powerful processor or fewer CPU cores, analysis may take longer, but should be feasible on standard personal computers.

Memory

Memory usage by Octava is dependent on memory usage by the background tools that conduct analysis (PLINK, BEAGLE). While gene/variant view should not be resource-intensive, association analysis, genome setup and genotype data processing may require a lot of memory and you may encounter “Out of memory” errors. If this happens, use the `Process externally` option in your input specifications.

Process externally

All the necessary files for your analysis will be packaged into a `.in.tar.gz` file found in the `data` directory. Copy this file to the `data` directory of a system with enough resources (e.g. high performance computing environment), with an instance of Octava set up via the `Conda installation` procedure.



Within the external instance, run `python process_external.py <mem>` (`<mem>` is optional, specifies memory in MB for the process to use). When finished, you will find the processed output file in the data directory. Copy the file (ending with `.out.tar.gz`) into the data directory of your main application instance. The file can be automatically loaded by clicking the `Load .tar.gz data` icon that will appear on the home page.

Setting up a dataset

Do not place the only copy of your files in the `data` directory. Files are modified during runtime.

Depending on the size of your input data, this may take on the order of hours (for terabyte-size data), but it is a one-time requirement.

Each dataset is organized as a folder in the `data` directory. The name of the folder is used as the name of the dataset; the **name can be any valid filename, and can include spaces, commas, periods**, etc. Place all your input files (see below) in this folder. This directory will be recognized when you go to `Add new dataset` .



With an input folder organized and located in the `data` directory, set up the dataset for use in the app by choosing your folder on the `Add new dataset` page. For each input file, specify the corresponding file, and `Submit` .

Input files and options

1. **Icon**
   - Optional, `PNG` or `JPEG` format, must be named `icon.<ext>`
   - Only visual, identifies a dataset on the home page
   - Ideally square aspect ratio, will be scaled to 128×128px
2. **Use files from existing dataset**
   - Useful for setting up a second `variant file` for the same genome
   - Genome-specific files are linked to the new directory, sample-specific files can be provided (`phenotype file` and `sample info file`)
3. **Variant file**
   - `VCF` or `BCF` format
   - Contains input genomic variant data
   - Uncompressed, or compressed using `bgzip` (not gzip)
   - Can be indexed beforehand using `tabix`

   Certain chromosome names will cause problems. **DO NOT** use the format `chr1,chr2,…` or similar. Use numbers (`1,2,…`) or **preferably**, word-like names (`Species.v1.0.chr1`). Replace chromosome names using `bcftools annotate --rename-chrs` , or use `awk` or `sed` commands. Additionally, chromosome names **cannot contain colons** (`:`).
4. **Sequence file**
   - `FASTA` format
   - DNA sequences of all chromosomes, contigs or scaffolds (available for all genome assemblies)
   - Uncompressed, or compressed using `bgzip` (not gzip)
   - Can be indexed beforehand using `samtools faidx` (both `.fai` and `.gzi` files are required)

   Names and lengths of chromosomes, scaffolds or contigs **must match** between the `sequence file` and the `variant file` . For mismatched names, rename them in one file or the other. For mismatched lengths, use the correct sequence file.

5. **Annotation file**
   - `GTF`, `GFF` or `GFF3` format
   - Annotation of genes and gene features (commonly available for most genome assemblies)
   - Uncompressed, or compressed using `bgzip` (not gzip)
   - Can be indexed beforehand using `tabix`

   Annotation file **must be sorted by position**. As an example way of sorting your file, use the command `sort -k1,1V -k4,4n -k5,5rn unsorted.gff > sorted.gff`.

   Functional annotations of genes in the annotation file (function, keywords, GO terms etc.) **will not be recognized and displayed** in `Gene/variant view`. Extract and provide them via the `gene info file` instead (see below).

6. **CDS file**
   - `FASTA` format, optional
   - Sequences of all annotated coding sequences, available to see in `Gene/variant view`
   - Uncompressed, or compressed using `bgzip` (not gzip)
   - Can be indexed beforehand using `samtools faidx` (both `.fai` and `.gzi` files are required)

7. **Protein file**
   - `FASTA` format, optional (required if you `Enable severity prediction` – see below)
   - Sequences of all annotated proteins, available to see in `Gene/variant view`
   - Uncompressed, or compressed using `bgzip` (not gzip)
   - Can be indexed beforehand using `samtools faidx` (both `.fai` and `.gzi` files are required)

8. **Phenotype file**
   - Optional, tab-delimited text file (`TXT` or `TSV` format)
   - Phenotype data for input samples, must be quantitative or binary (0/1) – categorical phenotype values are not supported
   - First row must contain trait names, first column must contain sample identifiers (not all samples from the `variant file` have to be present)

   In phenotype names, spaces and any of the characters `,:"'`[]{}<>?!^*#&$%\/|` will be replaced with an underscore `_`. If any duplicate names occur after replacement, duplicates will be renamed to `<name>.1`, `<name>.2` etc. **Avoid non-Latin characters.**

9. **Gene expression file**
   - Optional, tab-delimited text file (`TXT` or `TSV` format)
   - Contains expression data **per transcript (not gene)**
   - First row must contain run identifiers, first column must contain transcript identifiers (not all transcripts have to be present)

10. **Run info file**
    - Optional, tab-delimited text file (`TXT` or `TSV` format)
    - Contains metadata about gene expression runs
    - First row must contain metadata column names, first column must contain run identifiers

11. **Sample info file**
    - Optional, tab-delimited text file (`TXT` or `TSV` format)
    - Contains sample metadata

- First row must contain metadata column names, first column must contain sample identifiers (not all samples have to be present), **second column must contain sample names** (used in visualisation as human-friendly names), can be identical to sample identifiers

12. Functional info file

- Optional, tab-delimited text file (`TXT` or `TSV` format)
- Contains **transcript (not gene)** metadata (functional annotation, gene ontology, identifiers, …)
- First row must contain metadata column names, first column must contain transcript identifiers (not all transcripts have to be present)

13. BAM files

- Optional, BAM format
- Can be indexed beforehand using `samtools index`

Sample IDs **must match** between `variant` , `phenotype` and `sample info` files.

Transcript IDs **must match** between `annotation` , `gene expression` and `functional info` files.

Transcript IDs **must match** between `CDS` , `protein` and `annotation` files.

Files can be provided in the dataset directory as symbolic links.

If an uncompressed variant, annotation or sequence file is given as a symbolic link, the link will be removed and the file will be hard-copied to the dataset directory and compressed.

If a phenotype or sample information file is given as a symbolic link, the link will be removed and the file will be hard-copied to the dataset directory.

14. Annotate variants with SnpEff

- Provides additional information about variants in the VCF file
- Available in `Gene/variant view`
- As an optional check, provide the `CDS` and/or `Protein` sequences (see above)
- Creates a **separate VCF file** – if desired, remove the un-annotated VCF file manually

15. Enable severity prediction

- Reports predicted variant impact on protein function using PROVEAN
- Calculated on demand, available in `Gene/variant view`
- Requires the `Protein` sequences file (see above), and a BLAST database (downloaded or created using the `makeblastdb` command); **must be placed in a directory called `prot_db` within the root directory**

  Make sure to place your BLAST database in a location that **does not contain spaces** in the name of the directory **or any parent directories**.

  Severity prediction is intended to be used with a comprehensive protein database. Suggested databases are:
  - The BLAST non-redundant protein database (`nr` at https://ftp.ncbi.nih.gov/blast/db/);
  - UniProt reference clusters (`UniRef100` or `UniRef90` at https://www.uniprot.org/uniref/);
  - Or, depending on your species, a UniProt collection of protein sequences from a clade your species belongs to (with the benefit of being much smaller than the former two, while still retaining the most similar proteins to your target).

16. **Concatenate contigs**
    - Genomes with a large number of unplaced contigs will cause visualisation to slow down or fail. Specify these contigs to concatenate in plots as “Other”. This generally affects only visualisation, as well as the `Binary regression` model in `Association analysis` (see below).

Input VCF file will be filtered to remove:

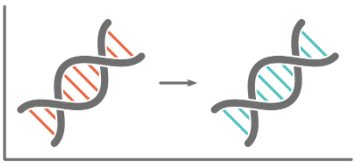
- Multiallelic variants (`--import-max-alleles 2`)
- Polyploid genotype calls (`--polyploid-mode missing`)
- Half-missing calls (`--vcf-half-call missing`)

using the specified PLINK2 flags (required by PLINK2 file formats). Additionally, all variant IDs will be set using `--set-all-var-ids @:#` .

After setting up a new dataset, it can be accessed on the home page, by clicking the corresponding icon and label.

Edit dataset

Upload additional trait data, sample metadata or association analysis cofactor data.



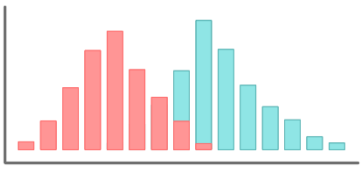
1. **Additional phenotype data, Additional sample metadata**
   - Same requirements as for `Setting up a new dataset`
   - New columns will be merged with the existing table by appending to the end
2. **New cofactor file**
   - Additional fixed-effect covariates for `Association analysis`, such as group membership, trial year or location
   - Must be numeric, encode categorical cofactors in one-hot encoding
   - First row must contain cofactor column names, first column must contain sample identifiers (not all samples have to be present)

     To use a cofactor file in association analysis with a given marker set, all samples from the marker set **must be present** in the cofactor file. Alternatively, filter out the extra samples.

Maximum size for any file is 50MB.

Phenotype/sample view

Browse and filter samples by metadata, compare traits and visualize sample distribution, process phenotype data and group traits or samples into groups.



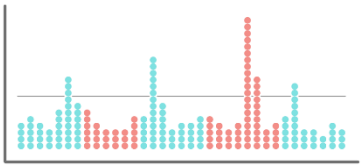
1. **Sample filtering**
   - View sample metadata table by clicking `Sample:`
   - View per-sample `Missingness` and `Heterozygosity` – the proportion of variant positions called as missing (out of total), and the proportion of variant positions called as heterozygous (out of all non-missing), respectively (calculated on the input variant data)
   - View phenotype values per sample for the selected `Trait`
   - ➕ save selection as `Sample group` , right-click group to see members
   - Any traits plotted will be shown per sample group
2. **Compare traits**
   - Select one or more traits or trait groups, see trait distribution and correlation
   - **Right-click a plot** to zoom into a single histogram or scatter plot (and right-click to zoom out)
   - For correlating 10 or more traits, plots a correlation matrix; **right-click column** to see sorted correlation to a single trait (and right-click to zoom out)
3. **Transform traits**
   - *Log$_e$ transform* – value converted to their log$_e$ value (trait must have only positive values)
   - *Box-Cox transform* – values converted to their x^n , where *n* yields the best approximation of a normal distribution of the transformed trait
   - *PCA transform* – calculates top *n* principal components of a group of traits, each component saved individually

     All traits **must be Z-score normalized** before calculating PCA transform.
   - *Z-score normalization* – values are scaled to have mean = 0 and standard deviation = 1
   - *Relatedness correction* – corrects trait values for sample relatedness via linear regression, given top *n* principal components or `Additional covariates` used as fixed effects, and/or a `Kinship` matrix used as random effects, taken from a chosen `Marker set`

     Correcting a trait for population structure is best done only for visualisation. It is carried out in two steps: input phenotype is corrected by linear regression given the fixed and/or random effects, and the residuals of the model are taken as corrected phenotype.
     Including full kinship will likely result in overcorrection; **use only principal components.**
   -
   - *Threshold cutoff* – removes values below `Minimum` or above `Maximum`
   - *Outlier removal* – removes values deviating more than `±SD from the mean`
   - *Make binary* – values below `Threshold` will become 0, values at or above will become 1

Association analysis

View genotype data statistics, apply filters to create curated marker sets, construct haplotype blocks. Carry out association analysis using various statistical models, visualize genome-wide significance and compare results, view linkage disequilibrium.



Genotype data preparation

1. **Plot**
   - View summary plots of various statistics of the genotype data
   - *Marker density* – number of markers in each 1Mbp window across the genome
   - *Linkage disequilibrium* – per-marker LD by chromosome, on a subset of markers uniformly distributed across the chromosome
   - Remaining plots – histograms of each metric, showing sample or variant distribution
2. **Subset from marker set**
   - Filtering and processing will be applied to an existing `Marker set` , instead of the raw VCF
   - Cannot be used on haplotype sets

   > To apply filters multiple times (and repeat quality control in between), apply filters once, **reapply to the intermediate set**, and delete the intermediate set when finished.
3. **Limit to samples**
   - Filter samples, given `Sample groups` created within `Phenotype/sample view`
4. **Limit to regions**
   - Filter markers by location, given `Regions` created within `Gene/variant view`
5. **Filter by indel size**
   - Filter variants by size (size of the largest allele is considered)
6. **Quality filters**
   - Filtering applied using PLINK2
   - Order controlled by PLINK2 (regions → variant size → variant heterozygosity → sample groups → sample heterozygosity → sample missingness → variant missingness → MAF → HWE)

   > To apply filters in a specific order, apply first filter to an intermediate set, apply second filter **to existing intermediate set** and delete the intermediate set when finished.
7. **Linkage disequilibrium (LD) pruning**
   - For each marker, removes all markers within `Window` if the linkage disequilibrium (r^2) between them is above `Threshold`
   - Applied **after imputation**
8. **Imputation**
   - Fills in missing genotype data and phases genotypes, using BEAGLE (default settings)
   - Applied **after quality and subset filters**
9. **Haplotype construction**
   - *LD-based* – constructed using PLINK1.9, based on the Gabriel, 2002 method
   - Blocks larger than `Max size` are split (setting a higher threshold will lead to higher memory usage and longer runtime)

- *SNP number window, Base-pair window* – groups consecutive markers into haplotype blocks with a given `Span` (in markers or base-pairs), moving by a given `Step` each time
- Applied **after LD pruning**

> Marker data will be genotyped based on haplotype blocks by **treating each haplotype in a block as a marker**, generating allele dosage data for each haplotype. Haplotypes are **filtered with a MAF threshold of 0.05 and unplaced markers are removed.**

Association analysis options

1. **Plot**
   - *Marker density* – number of markers in each 1Mbp window across the genome
   - *Block distribution* – only for haplotype marker sets; size and position of each haplotype block across the chromosomes
   - *Linkage disequilibrium* – per-marker LD by chromosome, on a subset of markers uniformly distributed across the chromosome
   - *Kinship matrix* – heatmap of the Hamming distance between samples based on genotype data
   - *Principal components* – scatter plot of principal components based on the genotype data, with the percentages of variance explained of the first fifteen principal components (can be coloured by `Sample group` , or filtered by `Sample group` and coloured by `Phenotype`
   - Remaining plots – histograms of each metric, showing sample or variant distribution

> Decimal numbers cannot be accurately represented in binary form and used by a computer. For example, the number 0.1 might be represented as 0.0999... In case you apply a *minor allele frequency* threshold of 0.1 (or in any similar filtering case), variants with MAF at exactly 0.1 might be evaluated as 0.09999… and plotted as being below the minimum threshold. This is **not an error in the filtering**, it is instead an artifact affecting only the histogram plot.

2. **Parameters**
   - Specify the desired `Marker set` , `Model` and `Traits` to run the analysis
   - Define population structure correction (models may have requirements to apply/not apply these)
   - Kinship and PC correction is applied using kinship matrix and principal components calculated from the specified Marker set by default, change this via `K/Q from`
   - Select a file with `Additional covariates` (provided via `Edit dataset`)

> To use a correction from a different marker set or from a cofactor file, all samples in the target marker set **must be present** in the cofactor file and in the source marker set. Alternatively, filter out the extra samples.

Models

- More information in the respective documentations and publications of the tools
- Default settings used, unless specified differently

1. **GLM, MLM, multi-trait MLM**
   - Via GEMMA (manual)
   - Kinship correction is not supported for GLM, required for MLM and multi-trait MLM

> Traits must be saved as a trait group to run multi-trait MLM.

2. **BLINK**
   - Via BLINK standalone C implementation (manual)
   - Kinship correction cannot be toggled, since it is always applied using internally calculated kinship matrix
3. **Binary regression**
   - Via REGENIE (manual)
   - If `Concatenated contigs` have been specified (see above), they are grouped under one pseudochromosome and both step 1 and step 2 of the analysis (see manual) run in this format (while setting `nauto` to the number of chromosomes without `Concatenated contigs`)
   - Number of markers for step 1 limited to 500,000; `bsize` set to 1000
4. **BayesB, BayesC**
   - Via GCTB (manual)
   - Filtering output marker by minor allele frequency and p-value (enabled by default) is disabled
   - Parameters `Burn-in iterations` , `Total iterations` and `Thinning` control the MCMC algorithm (more information)
5. **Random forest regression / classification**
   - Uses the python package `RangerForestRegressor`
   - Grows a random forest, with the *mtry* parameter specified using `Test markers` and the *ntree* parameter specified using `Trees per forest`
   - Uses **permutation importance** as the importance metric
   - Minimum node size set to `5`

     Correction for population structure in random forest is carried out in two steps: input phenotype is corrected by linear regression given the fixed and/or random effects, and the residuals of the model are taken as corrected phenotype.
     Including full kinship will likely result in overcorrection; **use only principal components.**

Experimental: to add custom association analysis models, by following the instructions and scheme in `modules/assoc/assoc_additional.py` .

Association analysis results

1. **View interactive Manhattan plots**
   - Specify the desired `Marker sets` , `Models` and `Traits` to plot
   - Choose a p-value threshold to plot
   - Plot the absolute value of the effect size instead of -$\log_{10}$(p-value)

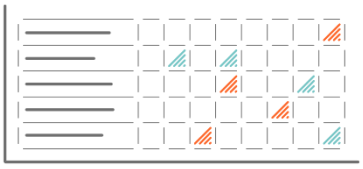
     Plotting the effect size is only available for GLM, MLM, Binary regression, BayesB and BayesC.
   - Interactive plot only shows top 5,000 significant markers; **view full plot and QQ-plot** (as static plots) by clicking `Show QC plots`

     The default threshold plotted is fixed to 8. Corrected threshold is not calculated, because results obtained through different methods can have different corrected thresholds.
     To calculate a Bonferroni-corrected threshold, i.e. `-log10(0.05/number of markers)` , find the number of markers by right-clicking your chosen `Marker set` .

2. **Plot a bubble plot**
   - Plot multiple Manhattan plots as a grouped bubble plot as a way of summarizing results
   - Plotted data will be grouped per `Window` , with a single bubble marker at the window midpoint
   - Bubble size represents the number of runs with significant association within the window (at or above `Significance`), bubble significance is the average significance of peak markers
   - `Num of traits` specifies a minimum number of runs for a point to be shown
   - To plot multiple groups, each group must be saved as a `Trait group`
3. **Current / selected region(s)**
   - Zoom into a locus within a single chromosome or **right-click a marker or a gene** to select a region to send to `Gene/variant view`
   - Highlight a `Region` or a `Gene group` , saved via `Gene/variant view`
4. **Linkage analysis**
   - **Left-click a marker** to view linkage disequilibrium (LD) to markers within `Window (kb)`
   - Set a minimum LD for markers to be plotted via `r² to peak` (for better performance), choose a different `Marker set` to extract markers from (e.g. unpruned / unfiltered markers)
5. **Extract QTLs**
   - All markers with significance at or above `Plot threshold` are extracted to an Excel table
   - QTLs are identified by:
     1. Taking the peak marker;
     2. Calculating LD to all markers within `Window (kb)` , taken from `Marker set` ; and
     3. Identifying the smallest region that includes all markers with r^2 at or above `r² to peak` ;
   - QTL identification is repeated as long as there are markers not part of already identified QTLs
   - The output includes the QTL span at a chosen `r² to peak` , as well as at $r^2 = 0.4$ and $r^2 = 0.8$
   - The `Effect size` and `Percentage of variance explained` are calculated using linear regression of the peak marker genotype and the trait, taken from the slope and the r^2 of the regression model, respectively
   - Calculated using both the raw trait and the trait corrected using linear regression with the fixed effect from the model (`Principal components` and/or `Additional covariates`) as model cofactors

Gene / variant view

Specify loci or genes of interest and view variant annotation, genotype data, SnpEff annotation, severity prediction and variant correlation with phenotype; view gene information, annotation and expression; view position of genes within the genome and of variants within a gene.



Input options

Many features of the **Gene / variant view** section depend on **SnpEff annotation** of the input VCF file. Annotate your VCF file beforehand or while `Setting up a dataset` for best results.

1. **Markers**
   - View variant data in `Raw VCF` (the unfiltered input VCF file), or in a filtered marker set created via the `Association analysis` section

     Genotype data (call quality, allelic depths, etc.) and SnpEff annotations **will not be visible** for the variants in a filtered marker set, and severity prediction will not be possible.
     Use this option (specifying a **filtered set instead of Raw VCF**) e.g. for viewing genotype calls per haplotype, or plotting phenotype correlation with a haplotype.

2. **Target**
   - Limit variant search to **only coding regions** – can speed up loading time if you are interested in coding variation only
   - Limit variant search to **only promoter regions** of all genes in the region / gene list (promoter region is defined as *n* base pairs upstream of transcript start, respecting the gene strand)

     Analyzing a locus of interest identified via Association analysis may better be done on the **Entire region**, rather than only **Coding regions**. Remember to change this option if desired.

3. **Variant impact**
   - Filter variants by the SnpEff-annotated `Annotation_Impact` attribute

High	Likely disruptive to the protein
Moderate	Might change protein effectiveness
Low	Likely to have no effect on the protein
Modifier	Usually non-coding, difficult to predict impact

   - Select `Don't show variants` if you are only interested in genes (greatly speeds up loading)

4. **Filter by**
   - `Sample group` : filter variants by sample group (requires a sample group to be created through **Phenotype/sample view**)
   - `Type` : filter variants by type (based on variant length or existing annotation generated by variant caller)
   - `Consequence` : filter variants by SnpEff-annotated `Annotation` attribute (Sequence Ontology nomenclature)

5. **Region(s) of interest**
   - `Region`, given the chromosome, start and end (end must be smaller than or equal to start)
   - `Gene ID`, or a list of IDs separated by space (see ? for an example Gene ID format)
   - `Coordinates`, in the format `chromosome` : `start` - `end` or `chromosome` - `position` ; or a list of such coordinates separated by space (start, end and position can include comma as a thousands separator)
   - `Gene attribute`, to find all genes containing `Query` within the chosen `Attribute` column in the `Gene info file`
6. **Variant quality**
   - `Missingness` : proportion of missing genotypes out of total
   - `Heterozygosity` : proportion of heterozygous genotypes out of all non-missing
   - `MAF` : frequency of the minor allele among all non-missing (for multiallelic variants: combined frequency of all non-major alleles)
   - Calculated **after filtering by sample**
   - Half-missing genotypes are treated as fully missing
7. **Skip**
   - Hide samples or variants that do not contain meaningful variation (see ? for more information)

When attempting to load more than 5,000,000 total genotypes (i.e. sample count * variant count), genotype data will be hidden from Variants table, and only variants will be displayed.

Results

1. **Variants**
   - Table of variant information and genotype calls
   - `Affected gene` – all the SnpEff-annotated `Gene_ID` attributes of the relevant annotations
   - `Missingness`, `Heterozygosity`, `Minor allele frequency` – calculated on the subset of genotypes **filtered for sample group selection**
   - `Annotation` – selected variant information: type + ordinal | sequence | *Annotation_Impact* | *Annotation* | *HGVS.p* | *ERRORS / WARNINGS / INFO* (attributes in italic are taken from SnpEff annotation)
   - `Severity score` – calculated for selected variants using the `Annotate severity` button and retained permanently once calculated, see below for more explanation
   - *Genotype calls* – missing allele denoted as `.`, reference allele denoted as `–`, alternative allele denoted as `type + ordinal`. Phasing (`|` or `/`) is retained in the call representation
   - Sample labels used in the table are the human-friendly **Name** (second column of the Sample info file), not the VCF **Identifier** (first column of the file)
   - See ? for explanation of **color-coding schemes**

     Genotype quality (for color-coding):
     If variant call has ***GQ*** attribute – quality score is the attribute value
     If variant call has ***PL*** attribute – quality score is the second lowest attribute value (lowest is typically zero)
     If variant call has ***GL*** attribute – quality score is the second lowest attribute value to the power of -10 (genotype likelihood converted to phred-scaled genotype likelihood)
     Quality score value is set to a maximum of 55 and mapped to a red-yellow-green color scale

> **Right-click functionality**:
> *Reference* – full reference sequence
> *Annotation* – all SnpEff-annotated attributes (see SnpEff manual for more explanation)
> *Severity score* – all attributes for severity prediction (see below for more explanation)
> *Genotype call* – all attributes for the call, as annotated by the variant caller

> **Severity prediction:**
> Calculated using PROVEAN. Severity score threshold of +/- 2.5 is used to assign the prediction of DELETERIOUS REF/ALT, respectively. The full prediction information (**right-click**) contains: variant type + ordinal, *Feature_ID*, *Gene_ID*, *Gene_Name*, *Feature_Type*, *Annotation*, *HGVS.p*, severity score, number of homologs used, number of clusters used, severity prediction (attributes in italic are taken from SnpEff annotation). See PROVEAN documentation for more information about the algorithm.

2. **Variant-phenotype correlation**
   - Boxplot of unique genotype values against the respective phenotype values by sample, for a chosen trait
   - **Left-click** a variant within the plot to **(un)select the variant** in the Variants table (select or unselect all variants using the `Select all` button of the Variants section to toggle between selecting variants in or out)

   > To find meaningful correlation between a variant and a trait, filter variants by minor allele frequency beforehand.

3. **Genes**
   - Table of gene information (position, strand), number of variants within (only the variants filtered based on search query), and additional annotations provided via the `Functional info` file

4. **Gene expression**
   - Boxplots of gene expression by attribute or group, for transcripts selected in the Genes table
   - Based on data provided via the Gene expression file
   - Gene expression runs can be filtered by run metadata; runs can be grouped and expression data plotted by groups
   - **Left-click** a gene within the plot to **(un)select the gene** in the Genes table (select or unselect all genes using the `ID` column dropdown to toggle between selecting variants in or out)

5. **Gene view**
   - Visualisation of the genome or specified regions of interest using `IGV.js`
   - Highlight and zoom into selected genes and variants using `View genes/variants`
   - Selected genes highlighted in red spans, selected variants highlighted in blue spans
   - Variant information is limited to position information (genotype calls are not shown) for performance reasons
   - For more information about the functionality, see IGV.js manual

6. **Sample clusters**
   - Table of samples clustered by identity across the queried variants

7. **Edit query**
   - Allows editing input parameters in-place
   - Go to selected genes or variants by choosing the appropriate `Format` and clicking